

Droplet Wall Interaction Tool: A Python Platform for Droplet Experiments and Qualitative Image Analysis

Kristina Mielke¹, Arif Rasim Can¹, and Andreas Jupke^{1,2}

¹ Fluid Process Engineering (AVT.FVT), RWTH Aachen University, 52074 Aachen, Germany 

² Institute for Bio- and Geosciences (IBG-2), Forschungszentrum Jülich GmbH, 52428 Jülich, Germany 

⁶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a

Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

TEst Droplet Wall Interaction Tool is a research-oriented, Python-based platform for qualitative image analysis of droplet experiments. Designed for academic and scientific workflows, it emphasizes reproducible batch processing and workflow automation for image analysis rather than hardware control. It integrates advanced image processing within a modern PySide6 (Qt for Python) interface and builds on NumPy, SciPy, pandas, and OpenCV ([Bradski, 2000](#); [Harris et al., 2020](#); [Qt for Python Team, 2025](#); [The pandas development team, 2025](#); [Virtanen et al., 2020](#)). Its modular architecture and batch processing capabilities enable high-throughput, reproducible analyses and facilitate rapid development of new workflows for surface science and fluid dynamics research.

Statement of need

Automated droplet experimentation and qualitative image analysis are essential in surface science, microfluidics, and materials research. Existing tools often lack integration, reproducibility, or extensibility. Droplet Wall Interaction Tool addresses these gaps by providing:

- Robust batch image analysis, including contact angle measurement, droplet diameter calculation, and velocity analysis.
- Automated droplet characterization using equivalent diameter formula ($D=\sqrt{(4A/\pi)}$) and area measurements.
- Visual area representation through transparent overlays for immediate visual feedback.
- Visual ROI selection and baseline adjustment with both numeric and graphical controls.
- Visual and numeric interfaces for region-of-interest (ROI) selection and baseline adjustment.
- Real-time logging and status indicators for transparency and troubleshooting.
- Robust handling of open or incomplete contours through conservative estimation methods. ([Mielke et al., 2025](#)) ([Pötter et al., 2026](#))

Droplet Wall Interaction Tool is designed for researchers who require a flexible, reproducible, and extensible platform for laboratory automation and qualitative analysis.

State of the Field

Software design

Research impact statement

AI usage disclosure

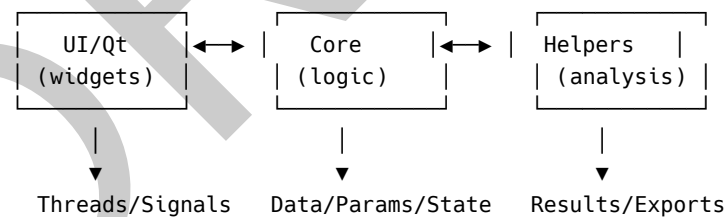
Features and implementation

Key Features

- **Analysis Tab:** Batch image processing, contact angle measurement, droplet area and diameter calculation.
- **Droplet Measurements:** Automatic calculation of droplet area and equivalent diameter using $D=\sqrt{(4A/\pi)}$ formula.
- **Visual ROI & Baseline:** Graphical and numeric interfaces for ROI selection and baseline adjustment.
- **Batch Processing:** Analysis of multiple datasets with progress visualization.
- **Velocity Analysis:** Advanced tools for droplet dynamics and time-series analysis.
- **Log Overlay:** Real-time logging with error/warning indicators.
- **Manual Testing:** Includes curated test images for rapid validation.
- **Modern UI:** Built with PySide6 for cross-platform compatibility and performance.
- **Modes:** free_sedimentation, contact_angle, channel, structured_packing (lazy-initialized in the app entrypoint).
- **Channel mode note:** Automatic baseline detection is currently disabled; channel overlays/metrics require externally provided baselines.

Architecture

Droplet Wall Interaction Tool follows a modular architecture:



The application is organized into core logic (`src/core.py`), UI widgets (`src/gui.py` and `src/widgets/`), helpers for image processing and analysis (`src/helpers/`), utilities (`src/utilities/` for image processing, logging, overlays, ROI management), and background threading (`src/utilities/threading.py`).

Image Analysis Pipeline

The image analysis pipeline is central to Droplet Wall Interaction Tool. It consists of:

1. **Crop and Rotate Image:** Preprocesses input images to ensure consistent ROI and orientation.
2. **Remove Background:** Enhances contrast and removes static artifacts.
3. **Detect Baselines:** Finds the baseline(s) where the droplet sits (mode-dependent).
4. **Contour Measurements:** Detects droplet contour and computes geometric features.
5. **Process Intersection Points:** Finds where the contour intersects the baseline(s).

- 66 6. **Contact Line Values:** Calculates the length/position of the contact line.
- 67 7. **Contact Angle:** Computes advancing/receding contact angles.
- 68 8. **Vertical Lines:** Detects vertical boundaries for wall contact analysis.
- 69 9. **Calculate Center Points:** Computes droplet center for velocity/tracking.
- 70 10. **Calculate Velocity:** Computes velocity from center point trajectory.
- 71 11. **Save Results (Excel):** Saves raw results to an Excel file with consistent formatting.
- 72 ▪ Implementation detail: per-frame overlays are saved under the selected folder; the Excel
- 73 file is written to the same folder as `results_raw.xlsx`.

74 Usage Example: Batch Analysis

- 75 1. Launch the application by running `python dwit.py`.
- 76 2. In the main window, open the **Analysis** tab.
- 77 3. For each experimental trial, ensure the data is organized in a separate folder containing
- 78 either:
 - 79 ▪ A video file (e.g., `trial1.mp4`), or
 - 80 ▪ A sequence of images (e.g., `frame_001.jpg`, `frame_002.jpg`, etc.)
- 81 4. Add the trial folders to the analysis queue using the **Add Folder** button.
- 82 5. Adjust analysis parameters (ROI, threshold, baseline) as needed.
- 83 6. Use **Preview** to verify settings or **Full Analysis** to process all queued trials.
- 84 7. Per-frame overlays are saved to the selected folder. The raw-results Excel is saved to
- 85 `<trial>/results_raw.xlsx`.

86 Citations

- 87 This work builds upon several key open-source libraries and tools:
- 88 ▪ NumPy ([Harris et al., 2020](#)) for efficient numerical computations
 - 89 ▪ Pandas ([The pandas development team, 2025](#)) for data manipulation and analysis
 - 90 ▪ SciPy ([Virtanen et al., 2020](#)) for scientific computing
 - 91 ▪ OpenCV ([Bradski, 2000](#)) for image processing and computer vision
 - 92 ▪ PySide6 ([Qt for Python Team, 2025](#)) for the graphical user interface
- 93 Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- 94 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
 95 Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk,
 96 M. H. van, Brett, M., Haldane, A., Del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant,
 97 T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 98
- 99 Mielke, K., Ickenroth, P., & Jupke, A. (2025). Experimental setup for understanding the
 100 influence of wall contact of structured packing material on drop sedimentation in liquid–liquid
 101 systems. *Industrial & Engineering Chemistry Research*, 64(36), 17685–17695. <https://doi.org/10.1021/acs.iecr.5c01952>
- 102
- 103 Pötter, P., Mielke, K., Polyakova, M., Jupke, A., & Grünewald, M. (2026). Toward reliable local
 104 fluid dynamic measurements in extraction: A film sensor approach in miniaturized setups.
 105 *Industrial & Engineering Chemistry Research*. <https://doi.org/10.1021/acs.iecr.5c05041>
- 106 Qt for Python Team. (2025). *PySide6: Official Python Bindings for Qt 6 (part of Qt for*
 107 *Python project)* (Version 6.9.1). The Qt Company. <https://doc.qt.io/qtforpython-6/>
- 108 The pandas development team. (2025). *Pandas-dev/pandas: pandas*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>
- 109
- 110 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
 111 Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M.,

112 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ...
113 SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
114 in python. *Nat. Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>

DRAFT